

Documento de Implementación del Prototipo

Introducción

Propósito

El propósito de este documento es describir el proceso de implementación del prototipo funcional de la plataforma de gestión de arriendo de vivienda urbana, documentando la metodología de desarrollo utilizada, las herramientas empleadas, las decisiones tomadas durante la construcción y los mecanismos de automatización que permitieron acelerar y estandarizar el flujo de trabajo. Este documento complementa el Documento de Especificación de Requisitos de Software (SRS) y el Diseño Arquitectónico y Funcional, cerrando el ciclo documental del proyecto al cubrir la fase de construcción.

Enfoque metodológico: Spec-Driven Development (SDD)

La implementación adoptó un enfoque de **Spec-Driven Development (SDD)**, una metodología de desarrollo asistida por agentes de IA donde cada funcionalidad se especifica formalmente antes de su implementación a través de tres artefactos estructurados:

1. **requirements.md**: Documento de requisitos con criterios de aceptación verificables
2. **design.md**: Documento de diseño técnico con decisiones de implementación
3. **tasks.md**: Lista de tareas ordenadas con checkpoints de verificación

Este flujo permite que el agente de IA trabaje de forma autónoma sobre tareas bien definidas, mientras el desarrollador mantiene control sobre el alcance, las decisiones de diseño y la calidad del resultado, bajo la hipótesis de reducir la subjetividad que queda por parte de la interpretación del agente, y por tanto la diferencia entre lo que se espera y lo que se obtiene. El ciclo SDD se repite iterativamente: se crea un spec, el agente implementa las tareas, se verifica el resultado y se documenta cualquier hallazgo post-implementación.

Ventajas del enfoque SDD en este proyecto

- **Trazabilidad**: cada línea de código puede rastrearse hasta un requisito específico
- **Iteración controlada**: el desarrollador define el "qué" y el agente ejecuta el "cómo"
- **Documentación viva**: los specs evolucionan con el proyecto y sirven como referencia técnica
- **Calidad incremental**: los checkpoints de verificación (build + tests) garantizan estabilidad entre tareas

Herramientas y Configuración del Entorno de Desarrollo

IDE y Agente de IA

El desarrollo se realizó utilizando **Kiro**, un entorno de desarrollo creado por **AWS**, el cual está potenciado por IA y trae **nativamente integrado el flujo de Spec-Driven Development (SDD)** como una de sus capacidades principales. A diferencia de otros IDEs con asistentes de IA que operan exclusivamente en modo conversacional, o que para implementar SDD requiere de configuraciones adicionales o de extensiones, Kiro

ofrece sesiones de tipo "Spec" donde el desarrollo se estructura formalmente a través de especificaciones, además de sesiones "Vibe" para exploración conversacional y correcciones puntuales.

La elección de Kiro como herramienta de desarrollo se fundamentó en:

- **SDD nativo:** El flujo de especificaciones (requirements → design → tasks) está integrado directamente en la interfaz, no requiere configuración adicional ni plugins externos
- **Steering files:** Soporte nativo para reglas de contexto que se inyectan automáticamente según patrones de archivos
- **Hooks:** Sistema de automatización basado en eventos del IDE que permite ejecutar acciones del agente sin intervención manual
- **Powers y MCPs:** Extensibilidad mediante Model Context Protocol para integrar herramientas externas

Kiro opera en dos modos de autonomía:

- **Autopilot:** el agente trabaja de forma autónoma completando tareas end-to-end
- **Supervised:** el agente solicita aprobación después de cada cambio

Para este proyecto se utilizó predominantemente el modo **Autopilot** durante la ejecución de tareas de specs, y el modo **Supervised** para correcciones puntuales y ajustes de diseño.

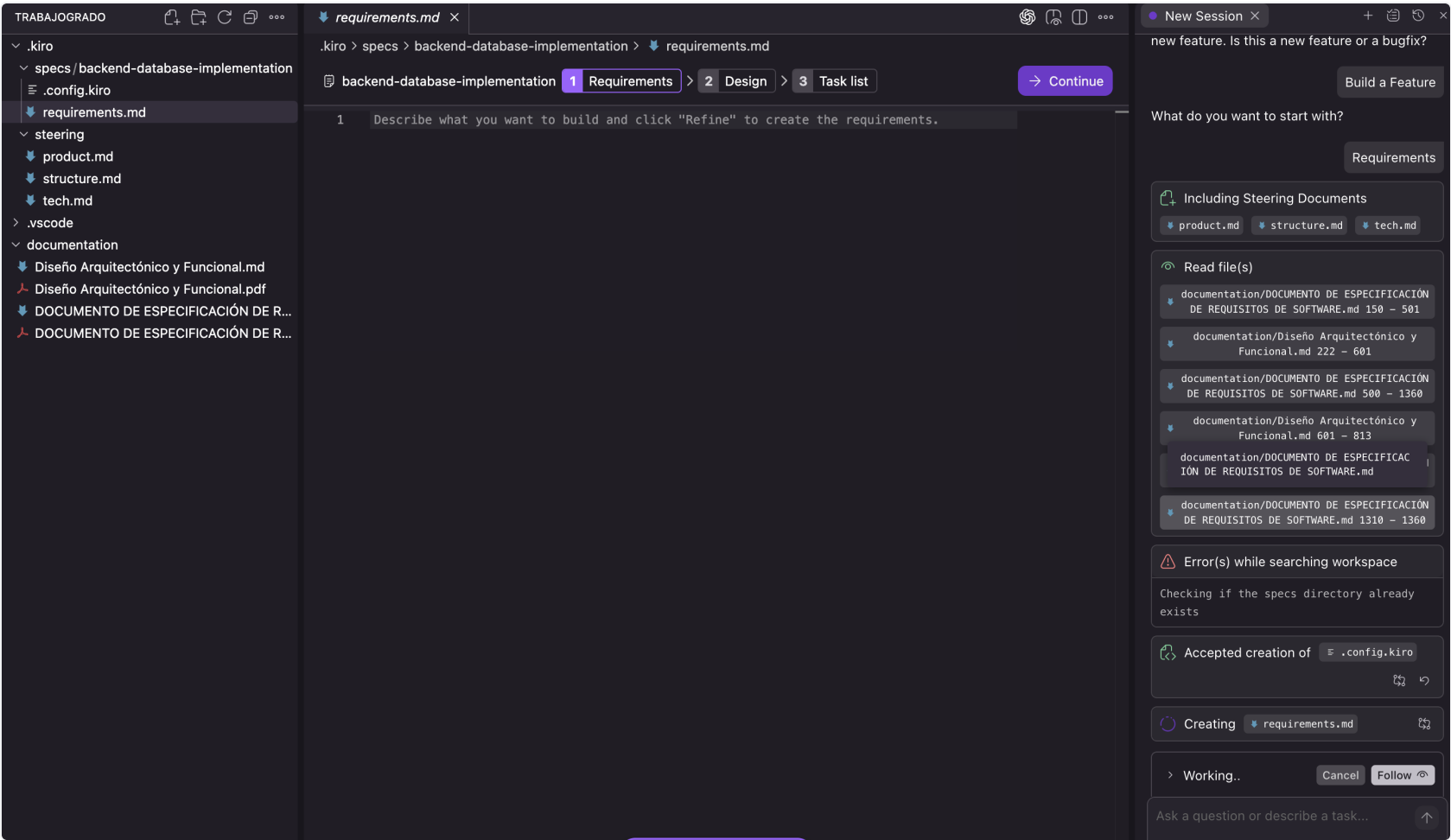
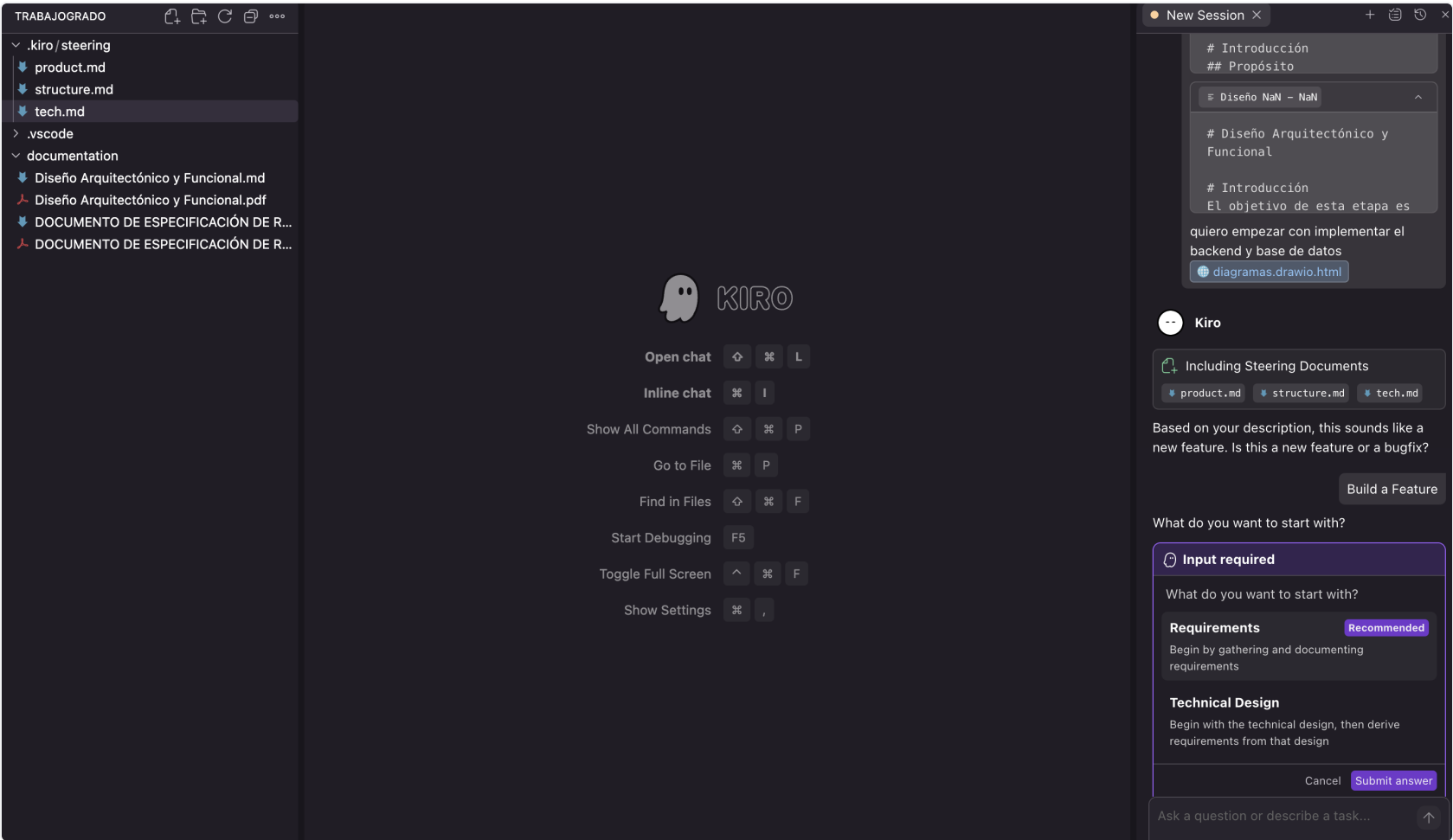
Flujos de SDD en Kiro

Kiro ofrece **tres flujos distintos** para crear specs, cada uno adaptado a un escenario de desarrollo diferente:

Flujo basado en Requerimientos (Requirements-first)

El flujo más completo. El agente parte de una descripción funcional del usuario y genera secuencialmente:

1. [requirements.md](#): Requisitos con criterios de aceptación verificables (formato WHEN/THEN/SHALL)
2. [design.md](#): Diseño técnico con decisiones de implementación, estructura de archivos y contratos de API
3. [tasks.md](#): Lista de tareas ordenadas con checkpoints de verificación (build + tests)



Este flujo se utilizó para la mayoría de features del proyecto donde el punto de partida era una necesidad funcional del usuario ya identificada en las etapas previas del prototipo.

Flujo basado en Diseño Técnico (Design-first)

Un flujo abreviado donde el desarrollador ya tiene claro el diseño técnico y quiere saltar directamente a la implementación. El agente genera:

1. [design.md](#): Diseño técnico detallado (proporcionado o co-creado con el agente)
2. [requirements.md](#) (*Opcionalmente*): Requisitos con criterios de aceptación verificables (formato WHEN/THEN/SHALL)
3. [tasks.md](#): Lista de tareas derivadas del diseño

The screenshot shows a code editor interface with a dark theme. At the top, there's a breadcrumb navigation: `ecs > aws-infrastructure-deployment > design.md > # Design Document: AWS Infrastructure Deployment > ## Architecture`. Below this, there are three tabs: `1 Design` (active), `2 Requirements`, and `3 Task list`. To the right of the tabs are buttons for `Sync Files` and `Continue`. The main editor area shows the content of `design.md`. It starts with a title `# Design Document: AWS Infrastructure Deployment` followed by a sub-header `## Overview`. The text describes an Infrastructure as Code (IaC) setup for a rental platform on AWS, listing components like NestJS backend, Next.js frontend, PostgreSQL database, Redis cache, and S3 object storage. It also mentions quality targets: LCP ≤ 2.5s, API response ≤ 800ms (p95), and availability ≥ 99.5%. The document recommends using AWS CDK with TypeScript. At the bottom of the visible text, there is a button labeled `Open Preview` with a magnifying glass icon.

Se puede omitir el documento de requisitos porque el contexto técnico ya está definido. Este flujo se utilizó para specs donde la solución técnica era clara desde el inicio.

Flujo de Corrección de Bugs (Bugfix)

Un flujo especializado para resolver defectos. El agente analiza el bug, documenta las condiciones que lo producen, las propiedades que deben cumplirse y las preservaciones (comportamientos existentes que no deben romperse), y genera:

1. [bugfix.md](#): Análisis del bug con el estado actual, esperado y estado que debe conservarse sin cambio
2. [design.md](#): Análisis del bug con Bug_Condition, Property y Preservation
3. [tasks.md](#): Tareas de corrección con tests de regresión

bugfix.md

.kiro > specs > ux-polish-fixes > bugfix.md > # Bugfix Requirements Document > ## Bug Analysis > ### Expected Beha

ux-polish-fixes1 Bugfix > 2 Design > 3 Task list

Sync FilesContinue

1# Bugfix Requirements Document

7## Bug Analysis

67### Unchanged Behavior (Regression Prevention)

70

713.2 WHEN the "Contactar arrendador" action succeeds THEN the system SHALL CONTINUE TO display the success message ("El contacto ha sido iniciado. El arrendador será notificado.") in a visible location

72

733.3 WHEN a landlord opens the contract creation wizard and the lease has a `monthlyRent` value THEN the system SHALL CONTINUE TO pre-fill the monthlyRent field correctly from the lease data

74

753.4 WHEN money amounts are formatted from raw digit strings (e.g., `1200000`) that do not already contain a `\$` symbol THEN the system SHALL CONTINUE TO display them correctly as `\$1.200.000`

76

773.5 WHEN the pagination component renders page number buttons and previous/next navigation THEN the system SHALL C

Open Preview

h correct spacing, touch targets (min 44px), and accessibility,

78

No incluye requirements.md porque el "requisito" es implícito: el sistema debe comportarse correctamente según su especificación original.

Clasificación de specs por flujo

Flujo	Specs
Requirements-first	backend-database-implementation , explore-properties-frontend , users-auth-frontend , landlord-portfolio-frontend , portfolio-figma-alignment , property-type-catalog , colombian-geo-catalog , landlord-modules-frontend , object-storage-implementation , portfolio-unit-listings-management , portfolio-contracts-management , contract-file-management , tenant-flows-frontend , platform-wide-improvements , multirole-notifications-frontend , listing-search-and-ux-enhancements , in-app-notifications-wiring
Design-first	aws-infrastructure-deployment
Bugfix	ux-polish-fixes , lease-lifecycle-status-sync

Los 17 specs de tipo Requirements-first siguieron el ciclo completo (requirements → design → tasks), generando documentación trazable desde la necesidad funcional hasta la implementación. Mientras que, los 2 specs de tipo Bugfix utilizaron el flujo especializado con análisis de condiciones de bug, propiedades esperadas y preservaciones, sin documento de requisitos separado. Por otro lado, solo se utilizó el flujo Design-first para la construcción de la infraestructura como código, dado que en gran medida este diseño ya estaba documentado.

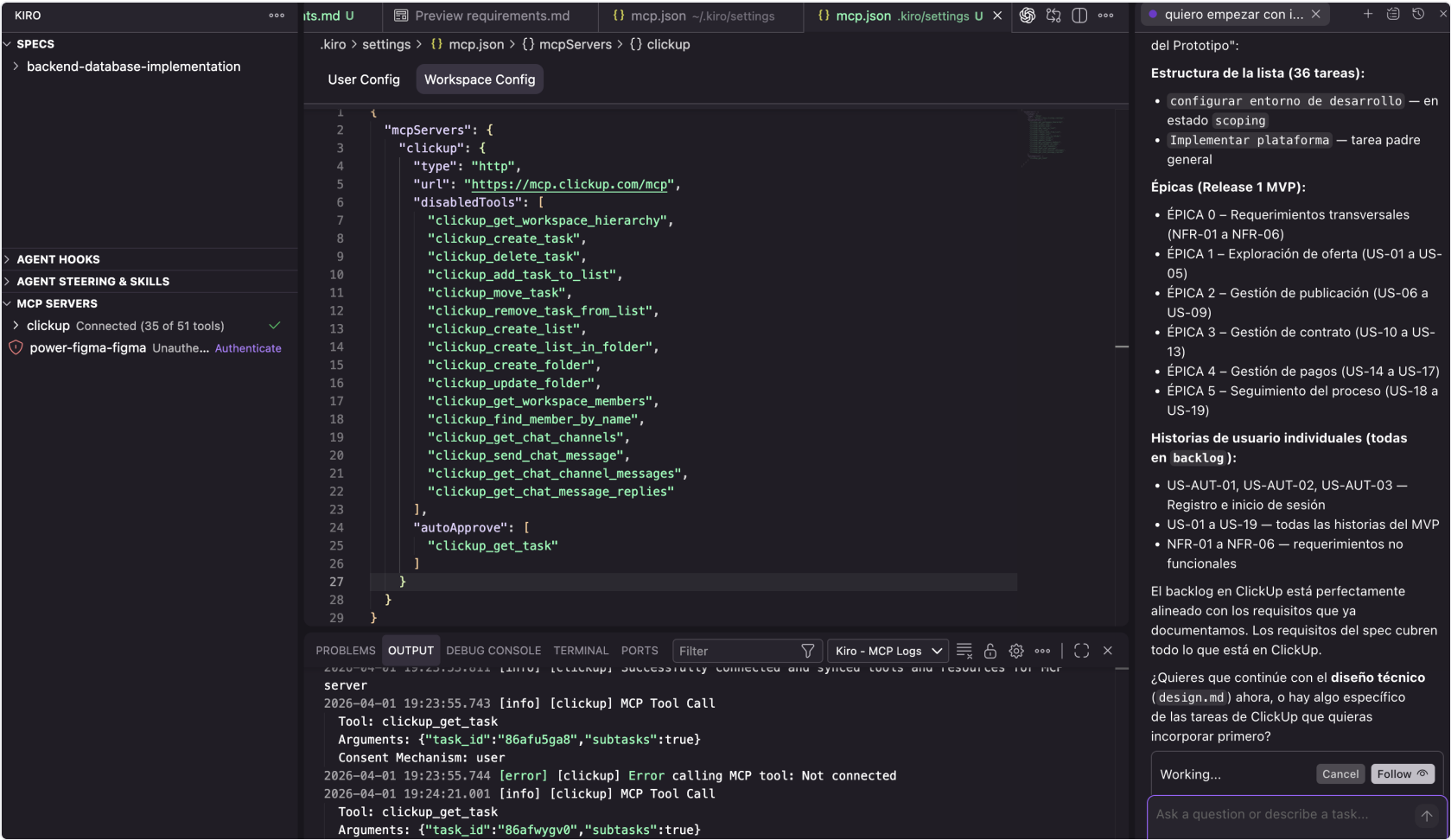
Model Context Protocol (MCP)

Se configuró un servidor MCP para integrar el flujo de desarrollo con herramientas externas:

ClickUp MCP

Propósito: Integrar la gestión de proyecto en ClickUp directamente con el flujo de desarrollo. Permite al agente:

- Consultar tareas y su estado (`clickup_get_task` , `clickup_search`)
- Actualizar el progreso de implementación (`clickup_update_task`)
- Documentar avances mediante comentarios (`clickup_create_task_comment`)
- Leer comentarios existentes para evitar duplicados (`clickup_get_task_comments`)



```
{
  "mcpServers": {
    "clickup": {
      "type": "http",
      "url": "https://mcp.clickup.com/mcp"
    }
  }
}
```

Configuración de seguridad: Se deshabilitaron herramientas destructivas o de alto riesgo (crear/eliminar tareas, mover tareas, gestionar miembros, enviar mensajes de chat) y se habilitó auto-aprobación para operaciones de lectura y comentarios.

Figma MCP (Power)

Se instaló el **Figma Power** como extensión de Kiro para validar la implementación frontend contra los diseños del sistema de diseño.

Propósito: Permitir al agente acceder a los archivos de Figma para:

- Extraer tokens de diseño (tipografía, colores, espaciado, radios de borde)

- Validar que los componentes implementados coincidan con las especificaciones visuales
- Verificar consistencia entre el código y el prototipo de baja fidelidad

Archivos de Figma referenciados:

- Librería del sistema de diseño (tokens, componentes base)
- Diseños de pantallas del prototipo

Steering Files (Reglas de Dirección)

Los steering files son documentos de contexto que se inyectan automáticamente en las interacciones con el agente de IA para guiar su comportamiento. Funcionan como "reglas del proyecto" que el agente debe seguir en toda ejecución, garantizando consistencia sin necesidad de repetir instrucciones manualmente.

Steering iniciales

Estos tres archivos se crearon al inicio del proyecto junto con la documentación de requisitos y diseño:

Stack tecnológico y decisiones técnicas (`tech.md`)

Define el stack completo (TypeScript full-stack, NestJS, Next.js, PostgreSQL, Prisma, Redis), la arquitectura (monolito modular con hexagonal por módulo), los patrones de seguridad (RBAC, PII encryption, circuit breaker), las convenciones de frontend (tipografía con `font-size: 62.5%`, tokens de diseño, patrones de componentes) y los comandos de desarrollo.

Impacto en SDD: Garantiza que cada spec generado y cada tarea implementada siga las mismas convenciones técnicas sin necesidad de especificarlas repetidamente.

Estructura del proyecto y convenciones (`structure.md`)

Documenta la organización de carpetas, la estructura hexagonal por módulo (`domain/`, `application/`, `infrastructure/`), las convenciones de naming (código en inglés, rutas en español, UI en español), las relaciones cross-schema y los componentes compartidos.

Impacto en SDD: El agente genera código que respeta la estructura existente y ubica archivos en las carpetas correctas automáticamente.

Contexto de producto y negocio (`product.md`)

Describe los usuarios objetivo (arrendadores adultos mayores con baja alfabetización digital, arrendatarios jóvenes), el alcance del MVP, los estados del proceso de arriendo y el contexto legal colombiano.

Impacto en SDD: Permite al agente tomar decisiones de UX informadas (simplicidad, baja carga cognitiva) y respetar el alcance del MVP sin agregar funcionalidades fuera de scope.

Steering agregados durante el desarrollo

`cross-schema.md`

Inclusión: Condicional, se activa solo cuando se leen archivos en `src/backend/modules/**`.

Objetivo: Documentar los patrones de comunicación cross-schema que emergieron durante la implementación: resolución de nombres de usuario (nunca mostrar UUIDs), descifrado de PII, referencias entre esquemas como campos `String` planos, y el patrón fire-and-forget para notificaciones.

Motivación: Durante la implementación de los módulos de contratos y arriendos, se detectaron errores recurrentes donde el agente intentaba hacer joins directos entre esquemas o mostraba UUIDs crudos en el frontend. Este steering eliminó esos errores al establecer reglas explícitas.

`frontend-patterns.md`

Inclusión: Condicional, se activa solo cuando se leen archivos en `src/frontend/**`.

Objetivo: Consolidar los patrones de componentes frontend que se establecieron durante las primeras iteraciones: tipografía con tokens personalizados (nunca usar `text-sm`, `text-lg` de Tailwind), estilo de botón primario, layout de páginas, navegación (hamburguesa vs. back arrow), y formato de moneda COP.

Motivación: El sistema de tipografía con `font-size: 62.5%` causaba que los tamaños por defecto de Tailwind resolvieran a valores incorrectos. Este steering previene ese error sistemáticamente.

`soft-delete.md`

Inclusión: Condicional, se activa solo cuando se leen archivos en `src/backend/**`.

Objetivo: Garantizar que toda query de lectura incluya `deleted_at: null` en el WHERE clause, usando las utilidades compartidas (`softDeleteFilter`, `softDeleteData()`, `withSoftDeleteFilter()`).

Motivación: Se detectaron bugs donde contadores de arriendos activos incluían registros eliminados, o donde el estado de una unidad se derivaba incorrectamente de leases cancelados. Este steering eliminó toda una categoría de bugs al hacer explícita la regla.

`spec-qa-stage.md`

Inclusión: Siempre (global).

Objetivo: Establecer que toda lista de tareas de un spec debe incluir una etapa final de QA manual donde el usuario prueba los flujos end-to-end y documenta hallazgos como nuevos requisitos en el mismo spec.

Motivación: Las primeras iteraciones de specs terminaban con "build passes" pero contenían problemas de UX no detectados (UUIDs visibles, traducciones faltantes, navegación inconsistente). Este steering formalizó el proceso de QA post-implementación.

Impacto en SDD: Transformó el flujo de un ciclo lineal (spec → implementar → done) a un ciclo con retroalimentación (spec → implementar → QA → documentar hallazgos → implementar fixes → done), mejorando significativamente la calidad del resultado final.

Hooks (Automatizaciones del Agente)

Los hooks son automatizaciones que ejecutan acciones del agente en respuesta a eventos del IDE. Permiten mantener documentación actualizada, estandarizar commits y validar diseño sin intervención manual.

Cronología de creación

Hooks de documentación automática

Los primeros hooks creados fueron los de actualización automática de READMEs:

Hook	Evento	Acción
<code>update-backend-readme</code>	<code>fileEdited</code> en <code>src/backend/**/*.ts</code> o <code>*.json</code>	Revisa el estado del backend y actualiza <code>src/backend/README.md</code>
<code>update-db-readme</code>	<code>fileEdited</code> en <code>db/**/*.prisma</code> , <code>*.ts</code> , <code>*.sql</code>	Revisa el schema Prisma y actualiza <code>db/README.md</code>
<code>update-frontend-readme</code>	<code>fileEdited</code> en <code>src/frontend/**/*.ts</code> , <code>*.tsx</code> , <code>*.json</code>	Revisa el frontend y actualiza <code>src/frontend/README.md</code>

Objetivo: Mantener la documentación técnica siempre sincronizada con el código sin esfuerzo manual. Cada vez que se edita un archivo de código, el agente revisa si el README correspondiente necesita actualizarse y lo hace automáticamente.

Impacto: Los READMEs del proyecto reflejan fielmente el estado actual del código en todo momento, sirviendo como documentación viva que no se desactualiza.

Hook de commits convencionales

```
{
  "name": "Auto Conventional Commit",
  "when": { "type": "postTaskExecution" },
  "then": {
    "type": "askAgent",
    "prompt": "Run git status, group changes by domain, create focused conventional commits, and push."
  }
}
```

Objetivo: Después de que el agente completa cada tarea de un spec, automáticamente agrupa los cambios por dominio/concern, crea commits con formato convencional (`feat/fix/chore/docs/refactor/test(scope): description`) y hace push al repositorio remoto.

Impacto: El historial de git es limpio, granular y trazable. Cada commit corresponde a un concern específico, facilitando la revisión y el rollback si es necesario.

Hook de validación Figma

```
{
  "name": "Figma Design Validation",
  "when": { "type": "userTriggered" },
  "then": {
    "type": "askAgent",
    "prompt": "Use Figma MCP to validate edited file against design system tokens..."
  }
}
```

Objetivo: Validar que los archivos frontend editados cumplan con el sistema de diseño definido en Figma. Verifica tipografía, colores, espaciado, radios de borde, sombras, targets táctiles y patrones de componentes.

Activación: Manual (el desarrollador lo ejecuta cuando quiere validar un componente contra Figma). Se eligió activación manual en lugar de automática para evitar llamadas excesivas al MCP de Figma durante iteraciones rápidas.

Hook de sincronización con ClickUp

```
{
  "name": "Sync Implementation Progress to ClickUp",
  "when": { "type": "userTriggered" },
  "then": {
    "type": "askAgent",
    "prompt": "Analyze codebase against ClickUp tasks in 'Implementar plataforma' milestone..."
  }
}
```

Objetivo: Analizar el estado actual del código y compararlo contra las tareas del milestone "Implementar plataforma" en ClickUp. Para cada historia de usuario (US) y requisito no funcional (NFR), determina si está implementado (✅), en progreso (🔧) o pendiente (⌚), y publica comentarios de progreso en cada tarea de ClickUp.

Características avanzadas:

- Deduplicación de comentarios (no publica si el estado no ha cambiado)
- Transiciones de estado unidireccionales (solo avanza, nunca retrocede)
- Resumen consolidado en la tarea padre del milestone

Impacto: Permite mantener sincronizado el estado de implementación entre el código y la herramienta de gestión de proyecto sin esfuerzo manual, facilitando la visibilidad del progreso para stakeholders.

Hook de infraestructura

```
{
  "name": "Update Infra README",
  "when": { "type": "fileEdited", "patterns": ["src/infra/**/*.ts", "src/infra/**/*.json", "src/infra/docker/*"] },
  "then": {
    "type": "askAgent",
    "prompt": "Review CDK infrastructure and update src/infra/README.md..."
  }
}
```

Objetivo: Mantener actualizado el README de infraestructura cuando se modifican stacks CDK, configuraciones o Dockerfiles.

Specs: Especificaciones de Implementación

Resumen de specs creados

Se crearon **20 specs** a lo largo del proyecto, cada uno cubriendo una funcionalidad o conjunto de mejoras específico. A continuación se presenta el orden cronológico de ejecución basado en la última modificación de sus archivos de tareas:

#	Spec	Fecha	Alcance
1	backend-database-implementation	16 abr	Backend completo: 8 módulos NestJS, schema Prisma, ETL, circuit breaker, tests
2	explore-properties-frontend	16 abr	Frontend de exploración: listado, filtros, detalle, galería de fotos
3	users-auth-frontend	17 abr	Frontend de autenticación: login, registro multi-paso, perfil, protección de rutas
4	landlord-portfolio-frontend	18 abr	Frontend del portafolio: listado, creación, edición, unidades
5	portfolio-figma-alignment	18 abr	Alineación visual con Figma: tokens, componentes, consistencia

6	property-type-catalog	18 abr	Catálogo de tipos de propiedad (backend + frontend)
7	colombian-geo-catalog	18 abr	Catálogo geográfico DANE: departamentos y ciudades
8	landlord-modules-frontend	19 abr	Módulos del arrendador: arriendos, contratos, publicación, contabilidad
9	object-storage-implementation	19 abr	Almacenamiento real S3: upload de fotos y contratos con presigned URLs
10	portfolio-unit-listings-management	23 abr	Gestión de publicaciones desde unidades del portafolio
11	portfolio-contracts-management	23 abr	Gestión de contratos desde el portafolio (crear, firmar, consultar)
12	contract-file-management	23 abr	Gestión de archivos de contrato: upload S3, reemplazo, eliminación
13	tenant-flows-frontend	23 abr	Flujos del arrendatario: arriendos, pagos, contratos
14	platform-wide-improvements	4 may	Mejoras transversales: soft delete, paginación, estadísticas
15	multirole-notifications-frontend	4 may	Notificaciones in-app, preferencias, gestión de roles
16	listing-search-and-ux-enhancements	4 may	Búsqueda por keywords, filtros backend-driven, landing page, rediseño
17	in-app-notifications-wiring	4 may	Wiring de notificaciones: adaptadores por módulo, fire-and-forget
18	ux-polish-fixes	5 may	Correcciones de UX post-QA: bugs, traducciones, navegación
19	lease-lifecycle-status-sync	6 may	Sincronización del ciclo de vida: tracking status,

			cancelación, derivación
20	aws-infrastructure-deployment	16 may	Infraestructura AWS: CDK, VPC, RDS, App Runner, CloudFront, WAF

Fases de implementación

Fase 1: Fundación (16 de abril)

El spec `backend-database-implementation` fue el más extenso y ambicioso, cubriendo la implementación completa del backend:

- Configuración del proyecto NestJS con Prisma, Redis y JWT
- Schema de base de datos con 8 esquemas PostgreSQL
- Implementación de los 8 módulos con arquitectura hexagonal
- Componentes transversales (guards, interceptors, circuit breaker, audit logger)
- Tests de propiedades (property-based testing) con fast-check
- Stubs para integraciones externas (pagos, firma, mensajería)

Fase 2: Frontend Core (16–18 de abril)

Tres specs construyeron la base del frontend:

- Exploración de inmuebles con filtros y paginación
- Autenticación con registro multi-paso y gestión de sesión JWT
- Portafolio del arrendador con CRUD de unidades

Fase 3: Catálogos y Alineación (18 de abril)

Specs de soporte que enriquecieron la experiencia:

- Alineación visual con los diseños de Figma
- Catálogo de tipos de propiedad para formularios
- Catálogo geográfico colombiano (33 departamentos, 1.122 municipios)

Fase 4: Módulos del Arrendador (19 de abril)

Implementación de los flujos completos del arrendador:

- Gestión de arriendos (crear, cancelar, historial)
- Gestión de contratos (wizard de 3 pasos, firma)
- Publicación de unidades con fotos
- Reportes contables con filtros de periodo

Fase 5: Gestión Avanzada (23 de abril)

Specs que conectaron los módulos entre sí:

- Publicaciones gestionadas desde las unidades del portafolio
- Contratos con upload real a S3 y presigned URLs

- Flujos completos del arrendatario (arriendos, pagos, contratos)

Fase 6: Calidad y Polish (4–6 de mayo)

Specs enfocados en calidad, consistencia y corrección de bugs:

- Soft delete transversal, paginación, estadísticas de portafolio
- Notificaciones in-app con preferencias de canales
- Búsqueda por keywords y filtros dinámicos
- Wiring de notificaciones entre módulos
- Correcciones de UX descubiertas en QA manual
- Sincronización del ciclo de vida del arriendo

Fase 7: Infraestructura (16 de mayo)

El spec final desplegó la infraestructura en AWS:

- 6 stacks CDK (Network, Data, CI, Compute, CDN, Monitoring)
- VPC con subnets públicas, privadas y aisladas
- RDS PostgreSQL + ElastiCache Redis + S3
- ECS Fargate para backend y frontend (containerizado)
- CloudFront + WAF para CDN y protección
- Monitoreo con CloudWatch, alarmas y dashboards

Estrategia de Testing

Property-Based Testing (PBT)

El backend utiliza **property-based testing** con la librería `fast-check` para verificar propiedades invariantes del sistema. Cada test está vinculado a un requisito específico del spec mediante comentarios de trazabilidad.

Áreas cubiertas:

- Sanitización de payloads maliciosos (XSS/SQL injection)
- Idempotencia de migraciones Prisma
- Restricciones de unicidad en la base de datos
- Round-trip de transformación ETL (RAW → curado)
- Invariantes de dominio por módulo (auth, portfolio, listings, contracts, payments, accounting, tracking, notifications)

Tests unitarios

Cada módulo incluye tests unitarios para:

- Funciones de validación puras (frontend y backend)
- Componentes compartidos (ProtectedRoute, StepIndicator)
- Helpers y utilidades (formatPrice, computePeriod, soft-delete utils)

Pruebas Funcionales y Validación Manual

La brecha entre especificación y resultado

A pesar de que la hipótesis central del proyecto es que el enfoque SDD entrega mejores resultados que el "vibe coding" (desarrollo conversacional sin estructura formal), la experiencia de implementación demostró que **sigue existiendo una brecha entre lo que se especifica y lo que se obtiene**. El agente de IA puede generar código que compila, pasa tests y cumple los criterios de aceptación formales, pero esto no garantiza que la experiencia del usuario sea la esperada.

Esta brecha se manifiesta principalmente en:

- **Aspectos visuales:** tipografía inconsistente, colores que no coinciden con el sistema de diseño, espaciados incorrectos, componentes que no se ven bien en ciertos tamaños de pantalla
- **Aspectos funcionales:** flujos que técnicamente funcionan pero resultan confusos para el usuario, información relevante que no se muestra en el momento adecuado, acciones importantes que quedan enterradas en la interfaz
- **Aspectos de contenido:** UUIDs crudos visibles al usuario, textos sin traducir al español, mensajes de error poco claros, estados sin etiqueta legible

Responsabilidad del desarrollador como usuario

Por esta razón, **sigue siendo responsabilidad del desarrollador revisar cada flujo como si fuera un usuario final**, interactuando con la aplicación de forma funcional después de cada ciclo de implementación. Esta revisión manual permite percibir oportunidades de mejora que ningún test automatizado puede detectar, porque se trata de juicios cualitativos sobre la experiencia de uso.

El proceso de validación manual seguido en este proyecto consistió en:

1. Completar la implementación de un spec (todas las tareas marcadas como done, build y tests passing)
2. Ejecutar la aplicación localmente y recorrer los flujos implementados como usuario final
3. Documentar cada hallazgo (bug visual, funcionalidad faltante, UX confusa) como un nuevo requisito
4. Agregar los hallazgos al spec existente en una sección "Post-Implementation Findings"
5. Implementar las correcciones como tareas adicionales del mismo spec

Origen del steering de QA

Fue precisamente a raíz de esta práctica recurrente de validación manual que surgió el steering `spec-qa-stage.md` (4 de mayo de 2026). Después de varias iteraciones donde los hallazgos post-implementación se documentaban de forma ad-hoc, se formalizó el proceso como una convención obligatoria: **toda lista de tareas de un spec debe incluir una etapa final de QA manual**.

Este steering transformó el flujo SDD de un ciclo lineal a un ciclo con retroalimentación explícita:

Sin QA stage:

spec → implementar → build passes → done ☒

Con QA stage:

spec → implementar → build passes → QA manual → documentar hallazgos → implementar fixes → done ✓

Los specs `ux-polish-fixes` y `lease-lifecycle-status-sync` son ejemplos directos de este proceso: ambos surgieron como resultado de la validación manual de flujos previamente implementados, donde se detectaron defectos que no eran visibles en los tests automatizados.

Integraciones Externas y Stubs del MVP

El MVP utiliza **adaptadores stub** para las tres integraciones externas que serán reemplazadas post-MVP:

Integración	Stub	Comportamiento
Firma electrónica	ESignatureProviderAdapter	Retorna un ID de firma mock; requiere webhook manual para completar
Pasarela de pagos	PaymentGatewayAdapter	Retorna APPROVED con URL de redirección mock; requiere webhook manual
Canal de mensajería	MessagingChannelAdapter	Registra notificaciones en consola del servidor

Para avanzar el estado de la aplicación durante testing, se documentó una **guía de testing con stubs** (`documentation/MVP-STUB-TESTING-GUIDE.md`) que incluye los comandos curl necesarios para simular los webhooks de firma y pago.

Patrones de Comunicación Inter-Módulo

Cross-Module Query Ports

Los módulos exponen interfaces de consulta para evitar queries SQL directos entre esquemas:

Token	Módulo	Métodos
PORTFOLIO_CROSS_MODULE_QUERY	landlord-portfolio	hasActiveLeases() , hasPortfoliosWithUnits() , hasActiveLeasesInPortfolios()
CONTRACTS_CROSS_MODULE_QUERY	contracts	hasActiveContractsAsRole()
PAYMENTS_CROSS_MODULE_QUERY	payments	hasPendingPayments()

Cross-Module Service Ports

Ports donde un módulo define la interfaz y otro provee la implementación:

Port	Define	Implementa	Propósito
PAYMENT_SCHEDULING_PORT	contracts	payments	Crea ScheduledPayment al completar firma
LISTING_DEACTIVATION_PORT	contracts	contracts (local)	Desactiva listing al firmar contrato

Notification Adapter Ports

Cada módulo que dispara notificaciones define un adaptador local que delega a `SendNotificationUseCase` :

Módulo	Eventos
contracts	CONTRACT_SIGNED , CONTRACT_UPLOADED , signing failed

payments	PAYMENT_RECEIVED
landlord-portfolio	LEASE_CREATED , LEASE_CANCELLED
property-listings	NEW_INTEREST
rental-tracking	CONTACT_INITIATED , CONTRACT_SIGNED , PAYMENT_RECEIVED

Infraestructura de Despliegue

Arquitectura AWS

La infraestructura se implementó con **AWS CDK** (Infrastructure as Code) organizada en 6 stacks modulares. La capa de cómputo utiliza **Amazon ECS Fargate** — un servicio de orquestación de contenedores serverless — con un **Application Load Balancer (ALB)** en subnets públicas para enrutamiento basado en path. CloudFront se ubica al frente para caching, protección WAF y terminación TLS.

Stack	Recursos
NetworkStack	VPC, subnets (3 tiers: públicas, privadas, aisladas), NAT Gateway, Security Groups (ECS, ALB, Data, EIC), EC2 Instance Connect Endpoint
DataStack	RDS PostgreSQL 16, ElastiCache Redis 7, S3 bucket, Secrets Manager
CiStack	ECR repositories, GitHub Actions IAM role
ComputeStack	ECS Cluster, ALB con path-based routing, Fargate services (backend + frontend), task definitions, IAM roles, auto-scaling
CdnStack	CloudFront distribution (ALB + S3 origins via OAC), WAF Web ACL, ACM certificate
MonitoringStack	CloudWatch alarms (ALB/ECS metrics), dashboards, log groups, SNS notifications

Flujo de requests

```
User → CloudFront (TLS) → WAF → ALB (HTTP, port 80)
  /api/*    → Backend Target Group → ECS Backend Service (private subnet)
  /assets/* → S3 (via Origin Access Control)
  default   → Frontend Target Group → ECS Frontend Service (private subnet)
```

Gestión de variables de entorno

- Todas las variables de entorno son gestionadas por CDK — **cero configuración manual en la consola AWS**:
- Variables no sensibles: inyectadas en las task definitions de ECS como environment variables
 - Secretos: almacenados en Secrets Manager y referenciados en las task definitions como secrets (resueltos en runtime por ECS)

Descubrimiento de requisitos emergentes

Un hallazgo significativo del proceso de implementación fue que **la interacción funcional con el prototipo ya concebido reveló oportunidades de mejora y flujos funcionales que no se habían considerado durante las**

fases de obtención de requisitos ni de diseño de la solución.

Ejemplos concretos de requisitos emergentes descubiertos durante la implementación:

- La necesidad de **desactivar automáticamente un listing cuando se firma el contrato** (no contemplado en el SRS original)
- La necesidad de **crear automáticamente un pago programado al completar la firma** (descubierto al probar el flujo end-to-end)
- La necesidad de **sincronizar el estado de tracking cuando se sube un contrato** (gap entre módulos no detectado en diseño)
- La necesidad de **derivar el estado "Ocupado" de una unidad solo cuando el contrato está firmado** (regla de negocio refinada por uso real)
- La necesidad de **mostrar información de contacto del arrendatario en la notificación de interés** (UX descubierta al usar la app como arrendador)

Estos descubrimientos son inherentes a cualquier proceso de desarrollo de software, pero el flujo acelerado mediante IA y en la medida que el desarrollador tenga contexto no solamente funcional sino también de la visión del negocio o estrategia, permite **detectarlos tempranamente**, en días o semanas en lugar de meses, gracias a que el prototipo funcional se materializa mucho antes de lo que permitiría un proceso tradicional.

Ventaja frente a metodologías tradicionales

En un flujo en cascada tradicional, estos requisitos emergentes se descubrirían típicamente en las fases de testing de integración o de aceptación del usuario, cuando el cronograma ya está comprometido y los cambios representan un **riesgo alto de incumplimiento**. La rigidez del proceso hace que cada hallazgo tardío se convierta en un cambio costoso que compite con la fecha de entrega.

En contraste, el flujo SDD asistido por IA permite:

1. **Materializar el prototipo funcional en días**, no en meses
2. **Descubrir gaps funcionales tempranamente** al interactuar con software real
3. **Iterar rápidamente** sobre los hallazgos sin comprometer el cronograma
4. **Documentar formalmente** cada descubrimiento como parte del spec (trazabilidad)
5. **Implementar correcciones** en el mismo ciclo de desarrollo, no como "deuda técnica"

Esta capacidad de descubrimiento temprano y corrección ágil es uno de los beneficios más significativos del enfoque adoptado, y refuerza la importancia de los procesos iterativos frente a los enfoques lineales para el desarrollo de productos digitales.

Conclusiones

- La implementación del prototipo permitió evidenciar la viabilidad del enfoque *Spec-Driven Development* (SDD) asistido por agentes de inteligencia artificial para el desarrollo de sistemas de complejidad media-alta, demostrando que el uso de especificaciones estructuradas y automatización puede acelerar significativamente la construcción de software manteniendo coherencia arquitectónica y funcional.
- Los resultados obtenidos demostraron que los *steering files* son un componente fundamental dentro del flujo SDD, debido a que permiten transferir reglas arquitectónicas, restricciones técnicas y convenciones de diseño al agente de IA. Gracias a ello fue posible reducir errores recurrentes relacionados con

consistencia visual, manejo de identificadores y reglas de persistencia, evidenciando que la calidad del resultado depende en gran medida de la claridad y precisión de las especificaciones entregadas.

- La experiencia de desarrollo permitió confirmar que la efectividad del enfoque SDD depende también de la calidad de las etapas previas de obtención de requerimientos y diseño arquitectónico. El trabajo realizado en el SRS y en el documento de diseño proporcionó un insumo sólido para la construcción de especificaciones y *steering files*, reduciendo la brecha entre el “qué” se debía construir y el “cómo” debía implementarse. Esto evidenció que el uso de IA sin un proceso previo riguroso de análisis y diseño puede incrementar el reproceso y las inconsistencias, mientras que una base documental clara potencia considerablemente la calidad y efectividad del desarrollo asistido por IA.
- La implementación permitió comprobar que, aunque los agentes de IA pueden generar código funcional y técnicamente válido, la etapa de validación manual continúa siendo indispensable para identificar problemas asociados a experiencia de usuario, navegación, consistencia visual y comportamiento funcional. Esto evidenció que el enfoque SDD no elimina la necesidad de QA humano, sino que transforma su rol hacia actividades de validación, refinamiento y aseguramiento de calidad orientadas al usuario final.
- A partir de las automatizaciones incorporadas mediante hooks y herramientas auxiliares, se evidenció una mejora significativa en la productividad del proceso de desarrollo, reduciendo trabajo repetitivo relacionado con documentación, validación de convenciones, estructuración de commits y sincronización con herramientas externas. Esto permitió mantener mayor consistencia y trazabilidad durante el desarrollo sin incrementar significativamente la carga operativa del proyecto.
- La arquitectura hexagonal implementada permitió validar la importancia de mantener separación entre dominio, puertos y adaptadores incluso en un contexto MVP, facilitando la incorporación progresiva de nuevas funcionalidades sin afectar considerablemente la lógica de negocio existente. Este resultado confirma que una arquitectura desacoplada favorece la mantenibilidad y evolución futura del sistema.
- La integración de MCPs con herramientas como ClickUp y Figma permitió mantener sincronización entre gestión del proyecto, diseño visual y desarrollo técnico dentro de un mismo flujo de trabajo, fortaleciendo la trazabilidad entre requerimientos, prototipos y funcionalidades implementadas. Esto evidenció el potencial de los ecosistemas integrados para reducir fricción entre las diferentes etapas del ciclo de desarrollo de software.
- El desarrollo temprano de un prototipo funcional permitió descubrir requerimientos, reglas de negocio y necesidades de interacción que no habían emergido durante las etapas iniciales de levantamiento de requisitos y diseño arquitectónico. Esto demostró que los ciclos acelerados de iteración asistidos por IA facilitan la detección temprana de vacíos funcionales y reducen el riesgo de desviaciones significativas en cronograma y alcance frente a enfoques tradicionales.
- En conjunto, la experiencia obtenida permitió concluir que el enfoque SDD asistido por agentes de inteligencia artificial no reemplaza el criterio humano dentro del desarrollo de software, sino que redefine el rol del desarrollador hacia actividades de supervisión, validación y toma de decisiones estratégicas. En consecuencia, el conocimiento de dominio, la capacidad de análisis y la comprensión de las necesidades del usuario continúan siendo factores fundamentales para garantizar la calidad y pertinencia de la solución desarrollada.